Build Your Own Cloud

Dev, DevOps & Ops

Bjørn Kristian Punsvik
November, 2025

```
bk@crayon:~$ whoami
--------------------
Name:       Bjørn Kristian Punsvik
Role:       Software Developer
Position:   Senior Consultant
Location:   Trondheim
Uptime:     30 years
Experience: 5 years
Education:  B.Sc. Computer Science, NTNU

bk@crayon:~$ ps -a
--------------------
PID   COMMAND                      TIME
1001  /usr/bin/linux-server-admin  9y
1002  /usr/bin/kubernetes          5y
1003  /usr/bin/aws                 2y
1004  /usr/bin/azure               3y
1005  /usr/bin/devops              2y
```

Development

The Twelve-Factor App

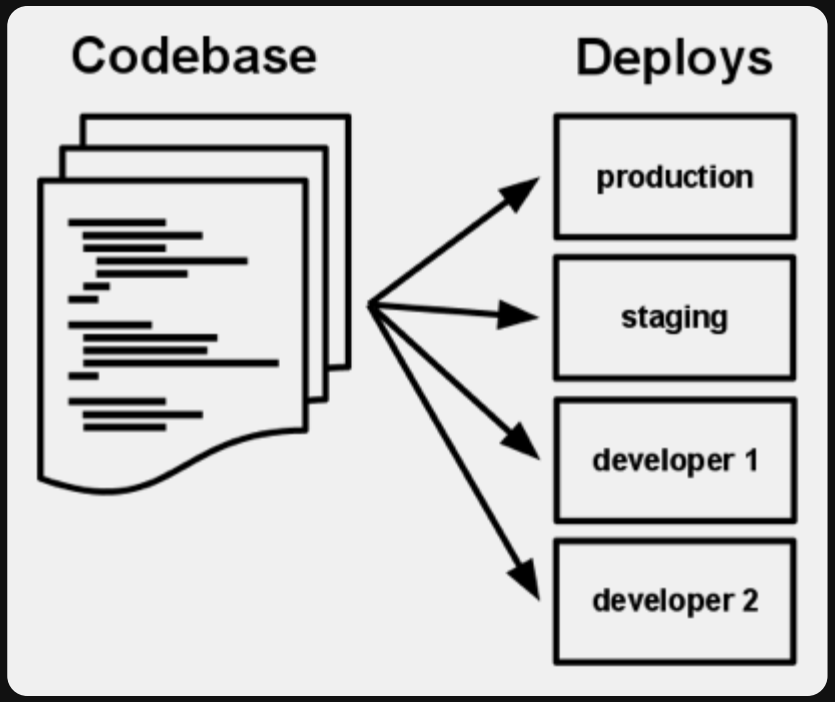
https://12factor.net

The Twelve-Factor App

I. Codebase
II. Dependencies
III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes

I. Codebase

II. Dependencies
III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes



One codebase tracked in revision control, many deploys

I. Codebase

II. Dependencies

III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes

A twelve-factor app never relies on implicit existence of system-wide packages.

Explicitly declare and isolate dependencies

I. Codebase
II. Dependencies

III. Config

IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes

An app's *config* is everything that is likely to vary between deploys (staging, production, developer environments, etc).

This includes:

- Resource handles to the database, Memcached, and other backing services
- Credentials to external services such as Amazon S3 or Twitter
- Per-deploy values such as the canonical hostname for the deploy

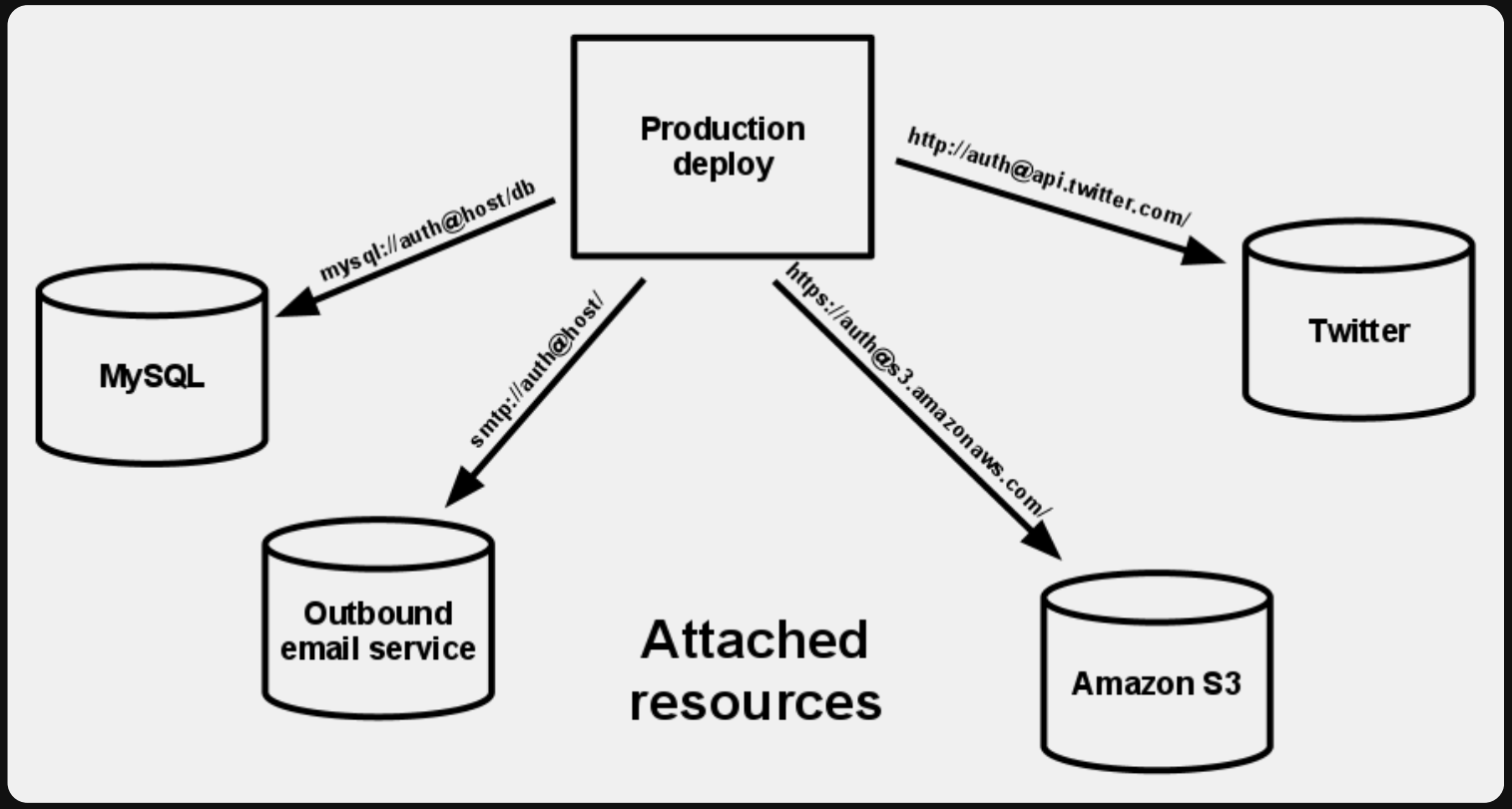
The twelve-factor app stores config in *environment variables* (often shortened to *env vars* or *env*).

Store config in the environment

I. Codebase
II. Dependencies
III. Config

IV. Backing services

V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes

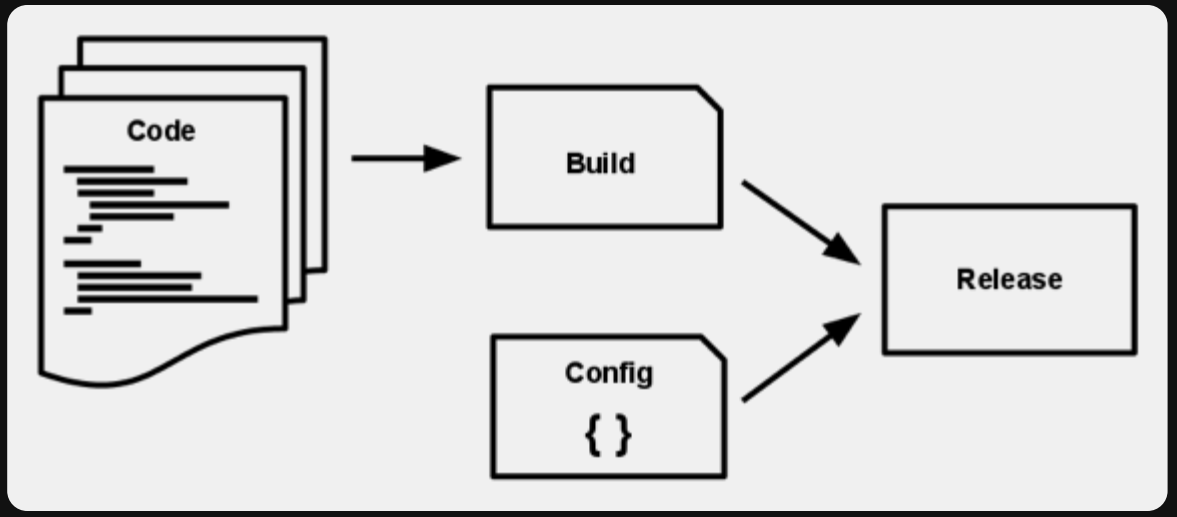


Treat backing services as attached resources

I. Codebase
II. Dependencies
III. Config
IV. Backing services

V. Build, release, run

VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes



Strictly separate build and run stages

I. Codebase
II. Dependencies
III. Config
IV. Backing services
V. Build, release, run

VI. Processes

The app is executed in the execution environment as one or more processes.

Twelve-factor processes are stateless and share-nothing. Any data that needs to persist must be stored in a stateful backing service, typically a database.

VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes

Execute the app as one or more stateless processes

I. Codebase
II. Dependencies
III. Config
IV. Backing services
V. Build, release, run
VI. Processes

VII. Port binding

VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes

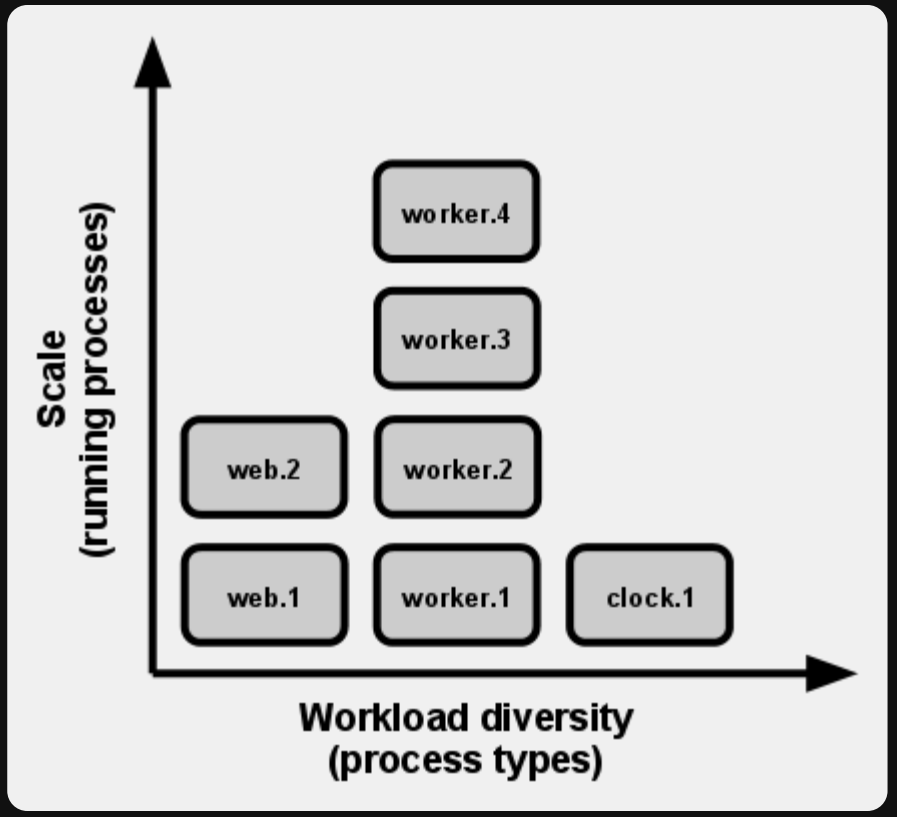
The twelve-factor app is completely *self-contained* and does not rely on runtime injection of a webserver into the execution environment to create a web-facing service. The web app exports HTTP as a service by *binding to a port*, and listening to requests coming in on that port.

Export services via port binding

I. Codebase
II. Dependencies
III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding

VIII. Concurrency

IX. Disposability
X. Dev/prod parity
XI. Logs
XII. Admin processes



Scale out via the process model

I. Codebase
II. Dependencies
III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency

The twelve-factor app's processes are disposable, meaning they can be started or stopped at a moment's notice.

Processes should strive to minimize startup time. Ideally, a process takes a few seconds from the time the launch command is executed until the process is up and ready to receive requests or jobs.

IX. Disposability

X. Dev/prod parity
XI. Logs
XII. Admin processes

Maximize robustness with fast startup and graceful shutdown

I. Codebase
II. Dependencies
III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability\

X. Dev/prod parity

XI. Logs
XII. Admin processes

The twelve-factor app is designed for continuous deployment by keeping the gap between development and production small.

	Traditional app	Twelve-factor app
Time between deploys	Weeks	Hours
Code authors vs code deployers	Different people	Same people
Dev vs production environments	Divergent	As similar as possible

Resists the urge to use different backing services between development and production

Keep development, staging, and production as similar as possible

I. Codebase
II. Dependencies
III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity\

XI. Logs

XII. Admin processes

Logs provide visibility into the behavior of a running app.

A twelve-factor app never concerns itself with routing or storage of its output stream.

Treat logs as event streams

I. Codebase
II. Dependencies
III. Config
IV. Backing services
V. Build, release, run
VI. Processes
VII. Port binding
VIII. Concurrency
IX. Disposability
X. Dev/prod parity
XI. Logs

XII. Admin processes

developers will often wish to do one-off administrative or maintenance tasks for the app, such as:

- Running database migrations
- Running a console (also known as a REPL shell) to run arbitrary code or inspect the app's models against the live database
- Running one-time scripts committed into the app's repo (e.g. `php scripts/fix_bad_records.php`)

Run admin/management tasks as one-off processes

Dotnet

10.0.100-rc.2

```
winget install Microsoft.DotNet.SDK.Preview
winget install Microsoft.DotNet.Runtime.Preview
winget install Microsoft.DotNet.AspNetCore.Preview
```

```
dotnet new webapp -lang C# -f net10.0 -n byoc -o . --exclude-launch-settings --no-https
```

.gitignore

```
# Downloaded content

# Built content

# User files

# Temp files

# Secrets
```

.gitignore

```
# Downloaded content

# Built content
bin
obj
# User files
.vscode
.idea
# Temp files
*.log
# Secrets
.env
```

.dockerignore

```
# Downloaded content

# Built content
bin
obj
# User files
.vscode
.idea
# Temp files
*.log
# Secrets
.env
# Non production files
```

.dockerignore

```
# Downloaded content

# Built content
bin
obj
# User files
.vscode
.idea
# Temp files
*.log
# Secrets
.env
# Non production files
appsettings.Development.json
**/launchSettings.json
```

Dockerfile

```
1 FROM
2 COPY
3 RUN
4 CMD
```

Dockerfile

```
1 FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine
2 COPY . .
3 RUN dotnet restore
4 RUN dotnet build -c Release
5 RUN dotnet test -c Release
6 RUN dotnet publish -c Release -o /dist
7 CMD
```

Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine
COPY . .
RUN dotnet restore
RUN dotnet build -c Release
RUN dotnet test -c Release
RUN dotnet publish -c Release -o /dist
ENV ASPNETCORE_URLS=http://+:8080
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080
CMD ["dotnet", "byoc.dll"]
```

Dockerfile

```dockerfile
FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine AS build
WORKDIR /src
COPY . .
RUN dotnet restore
RUN dotnet build -c Release
RUN dotnet test -c Release
RUN dotnet publish -c Release -o /dist

FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine
ENV ASPNETCORE_URLS=http://+:8080
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080
WORKDIR /app
COPY --from=build /dist .
CMD ["dotnet", "byoc.dll"]
```

Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine AS build
WORKDIR /src
COPY byoc.csproj .
RUN dotnet restore
COPY . .
RUN dotnet build -c Release
RUN dotnet test -c Release
RUN dotnet publish -c Release -o /dist

FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine
ENV ASPNETCORE_URLS=http://+:8080
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080
WORKDIR /app
COPY --from=build /dist .
CMD ["dotnet", "byoc.dll"]
```

Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0-alpine AS build
WORKDIR /src
COPY byoc.csproj .
RUN dotnet restore
COPY . .
RUN dotnet build -c Release
RUN dotnet test -c Release
RUN dotnet publish -c Release -o /dist

FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine
ENV ASPNETCORE_URLS=http://+:8080
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080
WORKDIR /app
COPY --from=build /dist .
CMD ["dotnet", "byoc.dll"]
```

Operations

CNCF

Meet us in Atlanta for KubeCon + CloudNativeCon North America · Nov 10-13 · REGISTER TODAY

CLOUD NATIVE COMPUTING FOUNDATION

About Projects Training Community Blog & News Join

WHO WE ARE

The Cloud Native Computing Foundation (CNCF) hosts critical components of the global technology infrastructure.

We bring together the world's top developers, end users, and vendors and run the largest open source developer conferences. CNCF is part of the nonprofit **Linux Foundation**.

OUR CHARTER

"CNCF's mission is to make cloud native computing ubiquitous..."

USEFUL LINKS

- Read about recent work and accomplishments in the **CNCF Annual Report**
- View our **FAQ**

https://cncf.io

Kubernetes (k8s)

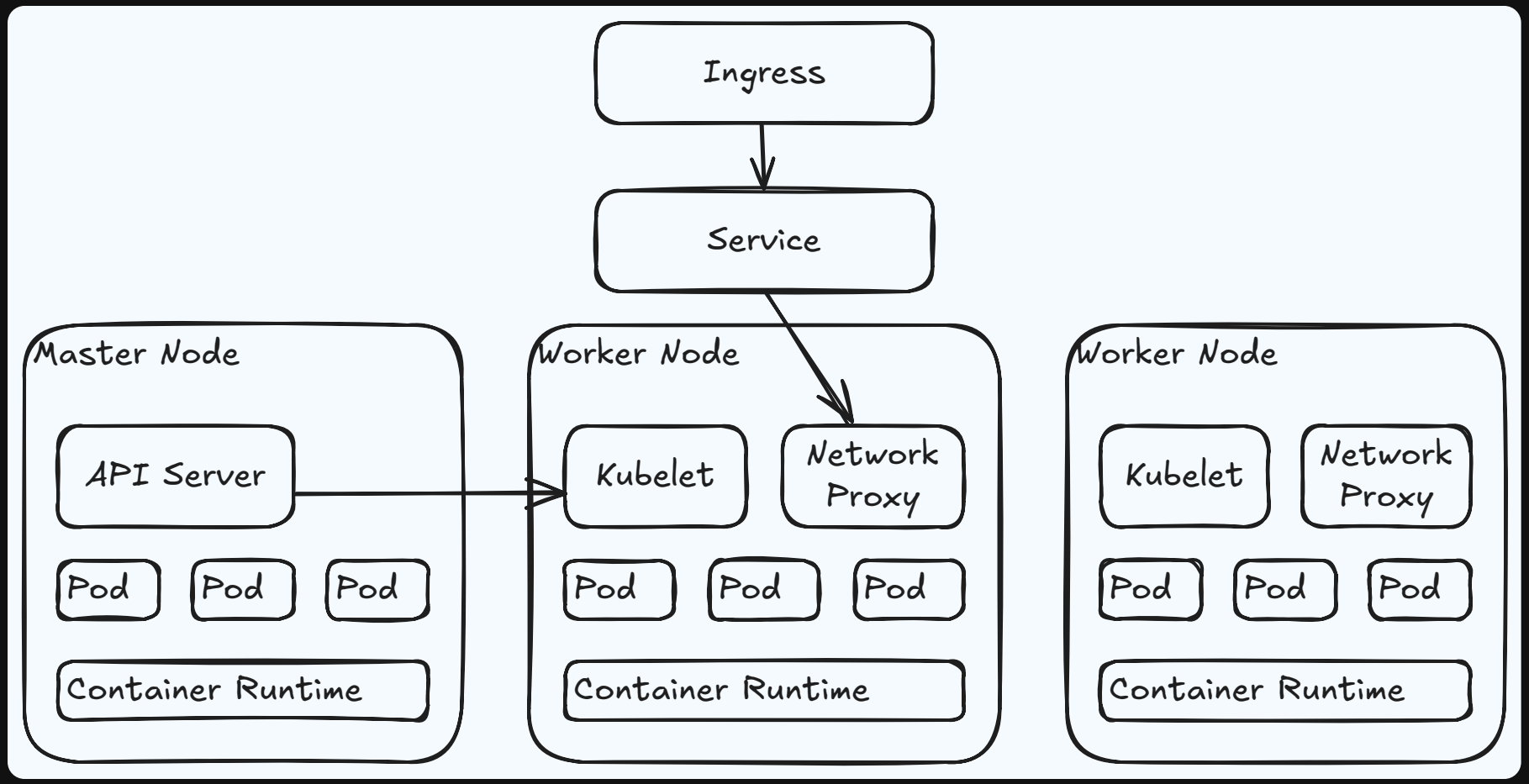
kubernetes

Kubernetes is an open-source system for automating deployment, scaling, and management of containerized applications

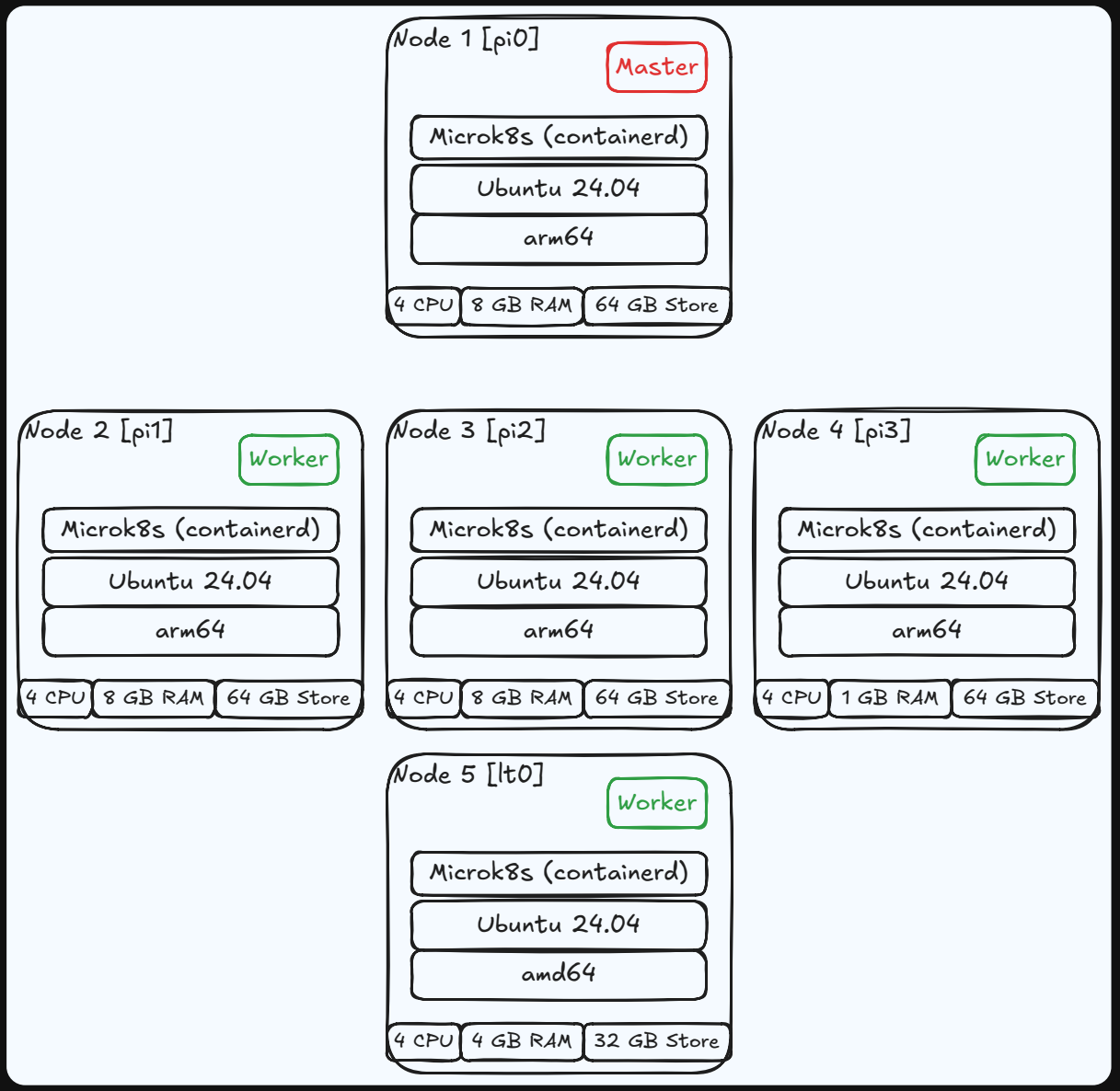
Kubernetes was accepted to CNCF on March 10, 2016 at the **Incubating** maturity level and then moved to the **Graduated** maturity level on March 6, 2018.

VISIT PROJECT WEBSITE

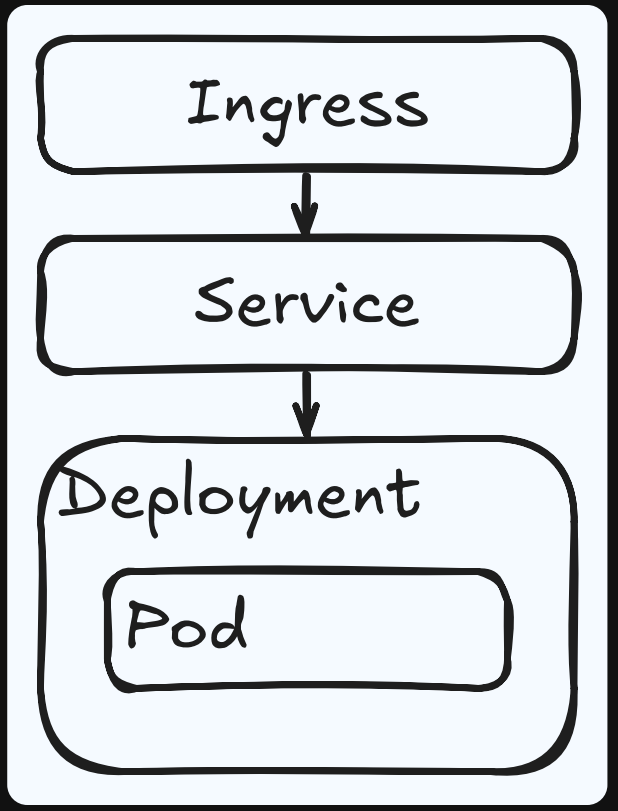
Kubernetes Architecture



Kubernetes Cluster



Kubernetes Deployment



Kubernetes manifest

`k8s.yml` - Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: crayon
  name: build-your-own-cloud
spec:
  replicas: 1
  selector:
    matchLabels:
      app: build-your-own-cloud
  template:
    metadata:
      labels:
        app: build-your-own-cloud
        version: IMAGE_LABEL
    spec:
      imagePullSecrets:
        - name: regcred
      containers:
        - name: build-your-own-cloud
          image: ghcr.io/punsvikcloud/build-your-own-cloud:IMAGE_LABEL
          imagePullPolicy: Always
          resources:
            limits:
              memory: "128Mi"
              cpu: "500m"
          ports:
            - containerPort: 8080
```

Kubernetes manifest

`k8s.yml` - **Service**

```yaml
apiVersion: v1
kind: Service
metadata:
  namespace: crayon
  name: crayon-service
spec:
  selector:
    app: build-your-own-cloud
  ports:
    - port: 8080
```

Kubernetes manifest

`k8s.yml` - Ingress

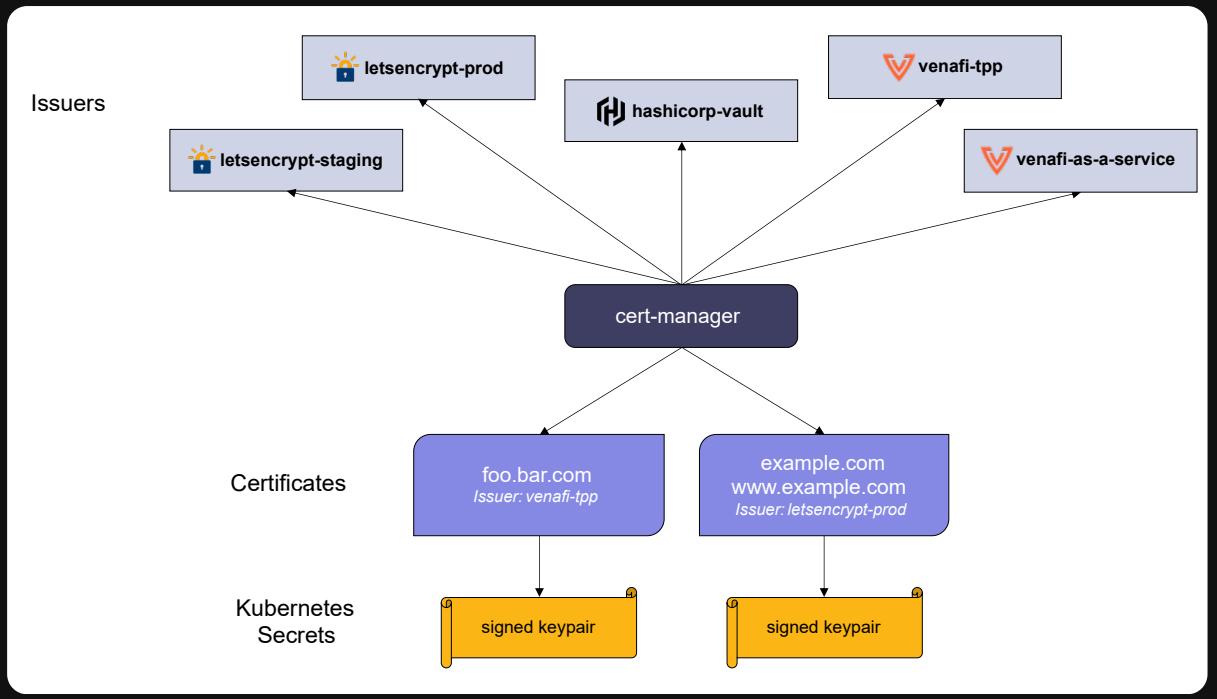
```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: crayon
  name: crayon-ingress
  labels:
    name: crayon-ingress
  annotations:
    cert-manager.io/cluster-issuer: lets-encrypt
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - crayon.punsvik.net
      secretName: tls-secret
  rules:
    - host: crayon.punsvik.net
      http:
        paths:
          - pathType: Prefix
            path: "/"
            backend:
              service:
                name: crayon-service
                port:
                  number: 8080
```

Kubernetes manifest

`k8s.yml` - Ingress

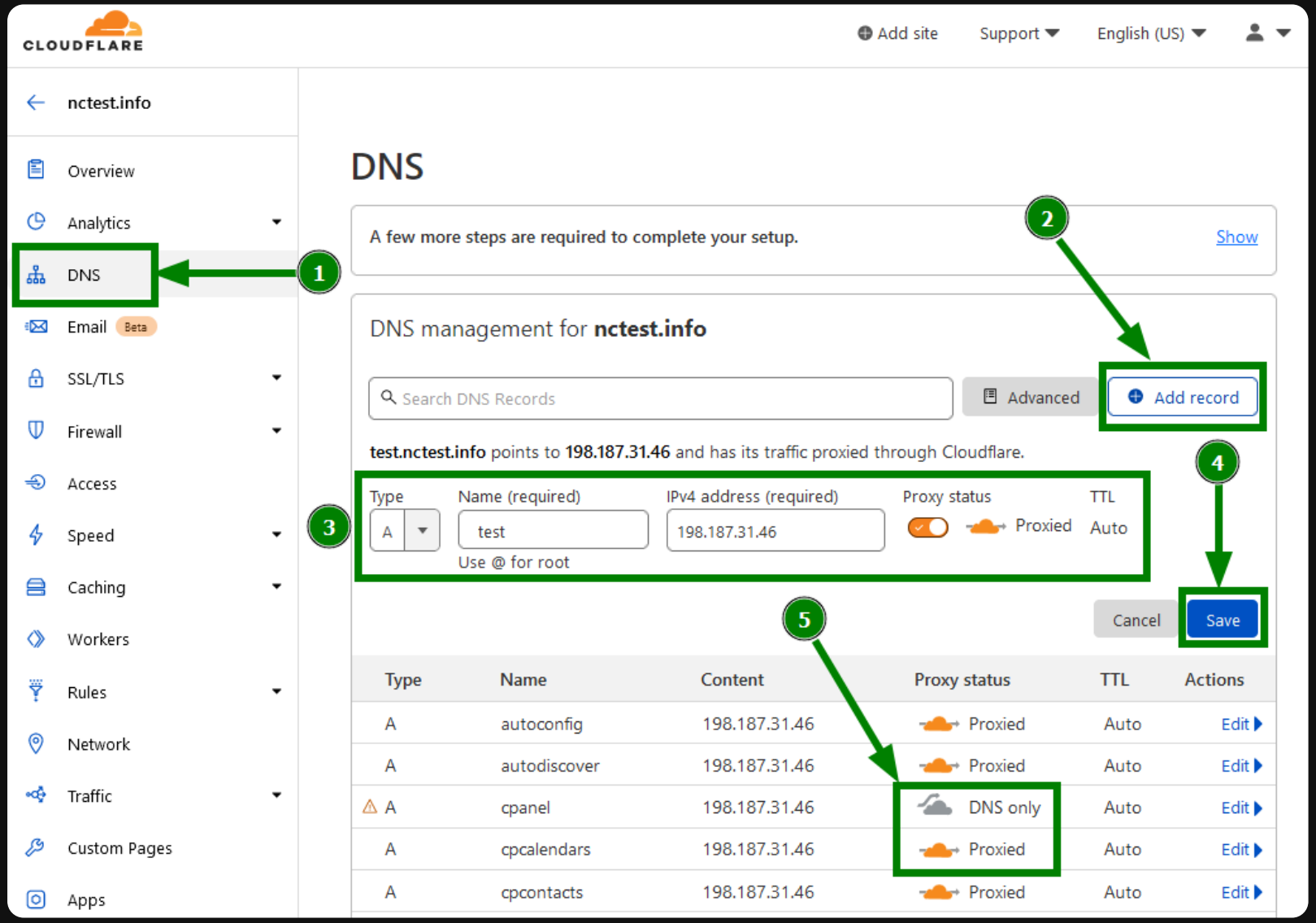
```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: crayon
  name: crayon-ingress
  labels:
    name: crayon-ingress
  annotations:
    cert-manager.io/cluster-issuer: lets-encrypt
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - crayon.punsvik.net
      secretName: tls-secret
  rules:
    - host: crayon.punsvik.net
      http:
        paths:
          - pathType: Prefix
            path: "/"
            backend:
              service:
                name: crayon-service
                port:
                  number: 8080
```

cert-manager



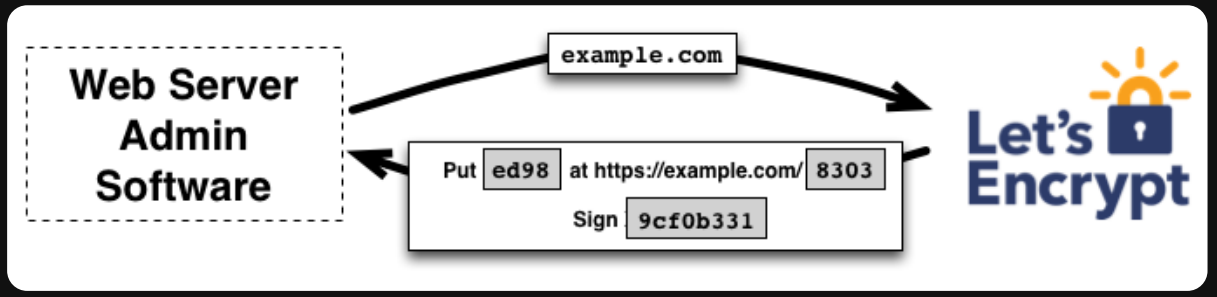
https://github.com/cert-manager/cert-manager

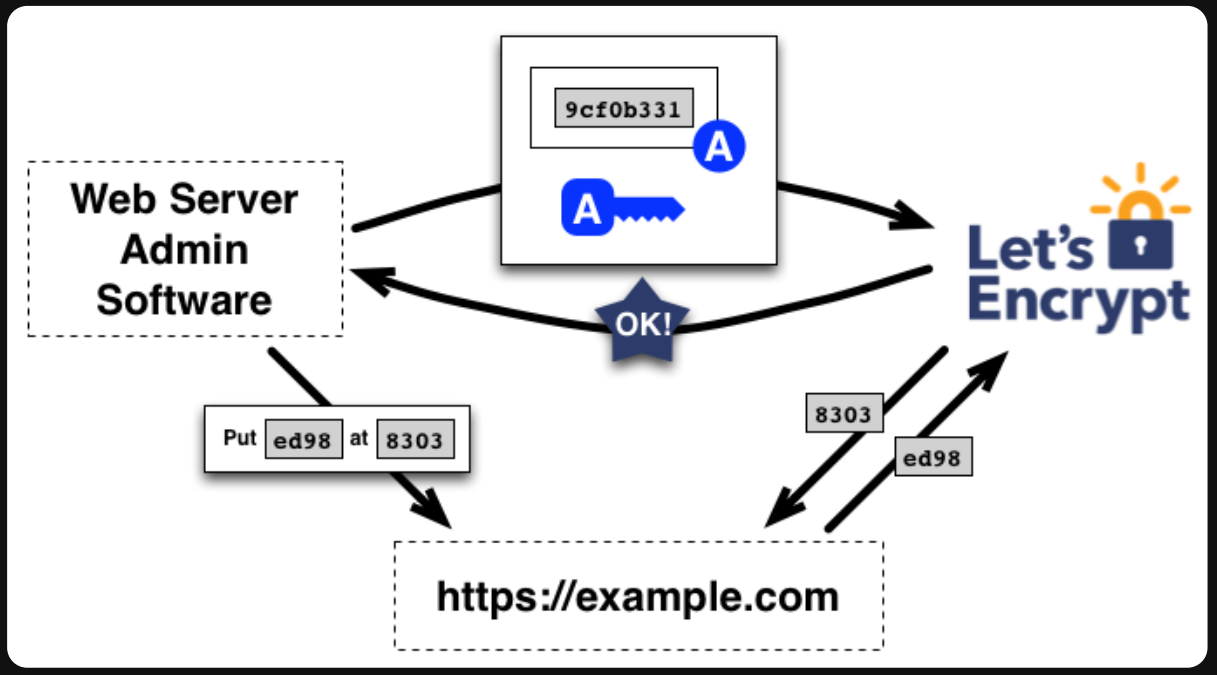
Cloudflare DNS



Let's Encrypt

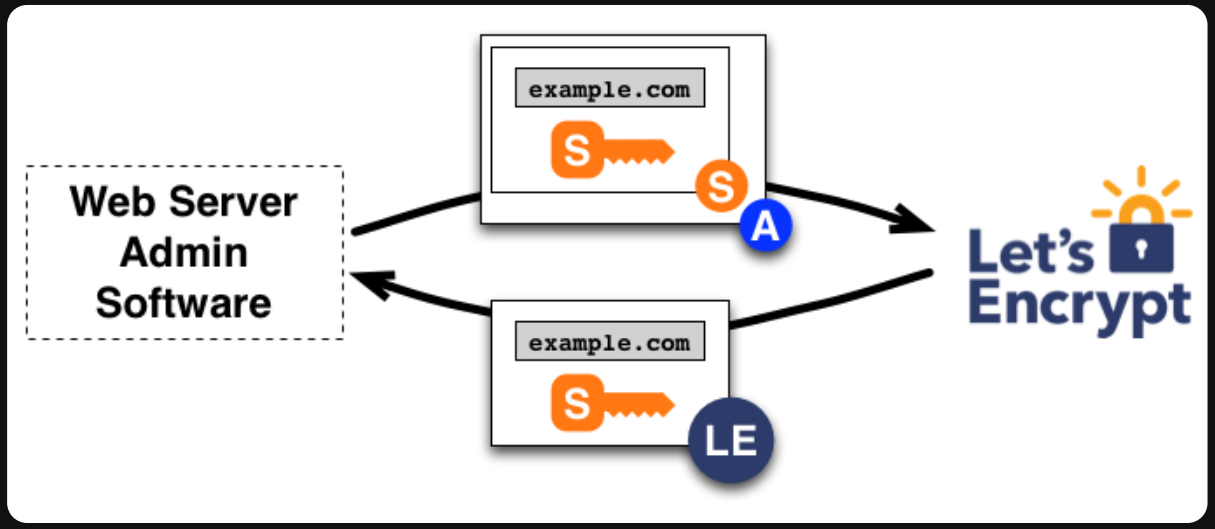
Domain Validation



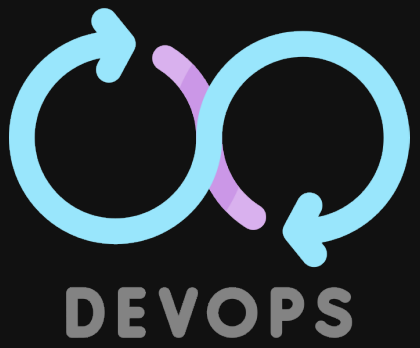


Let's Encrypt

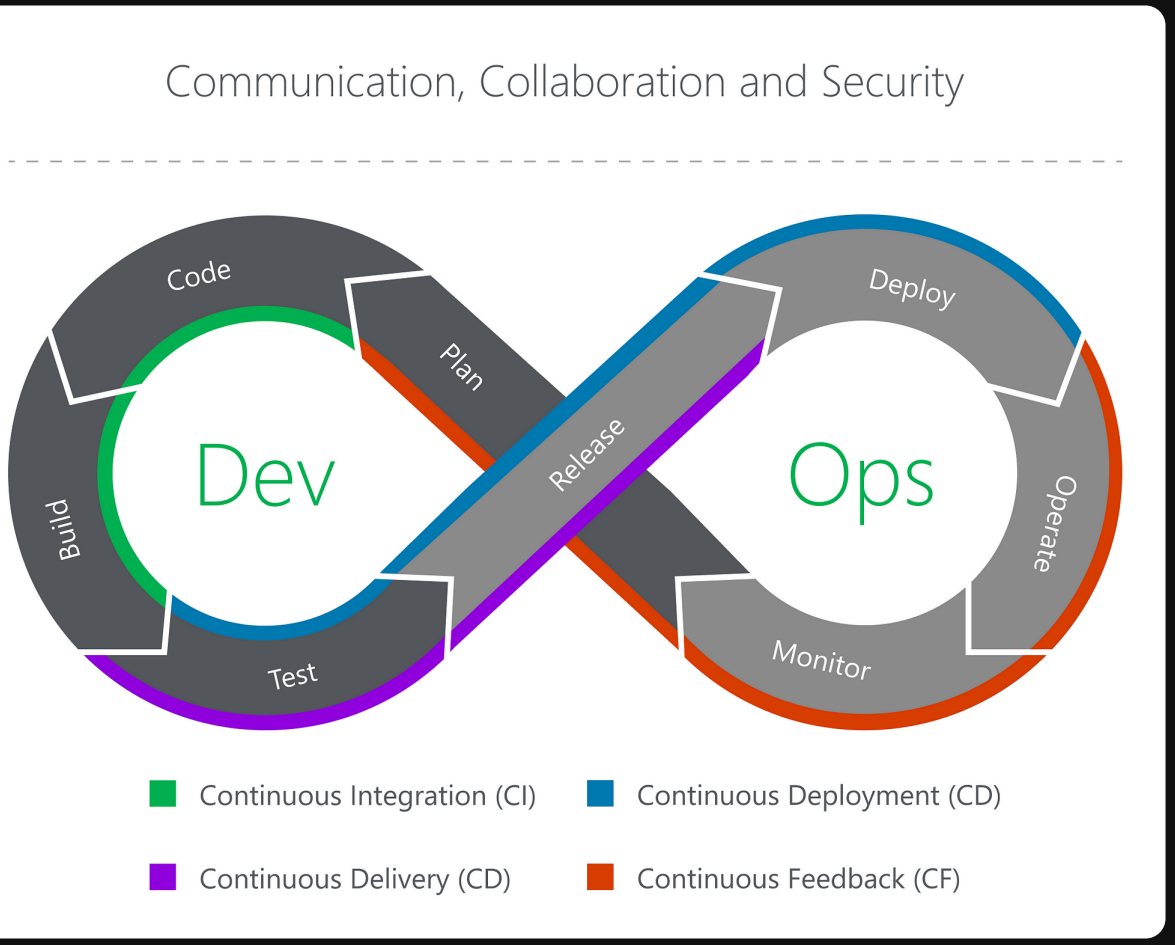
Certificate Issuance



DevOps

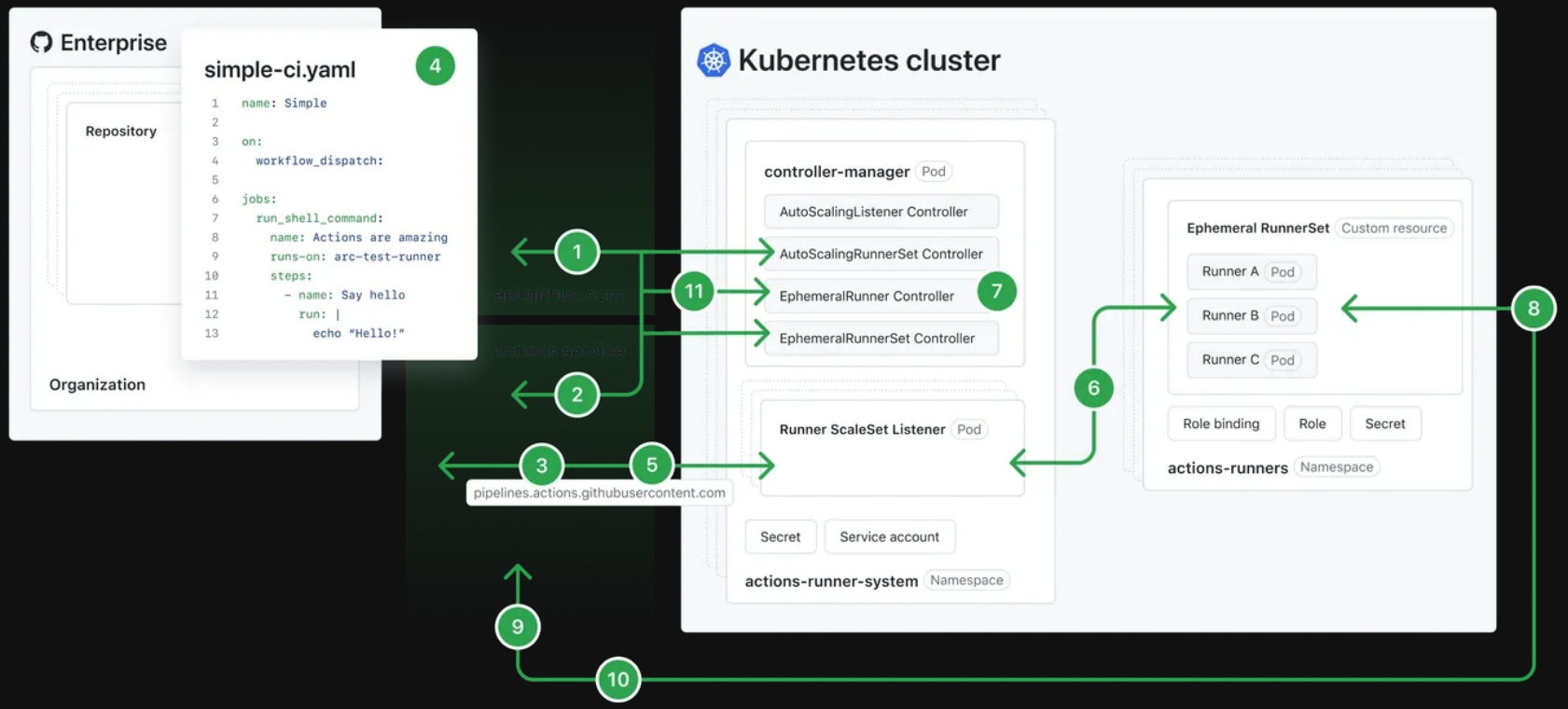


DevOps

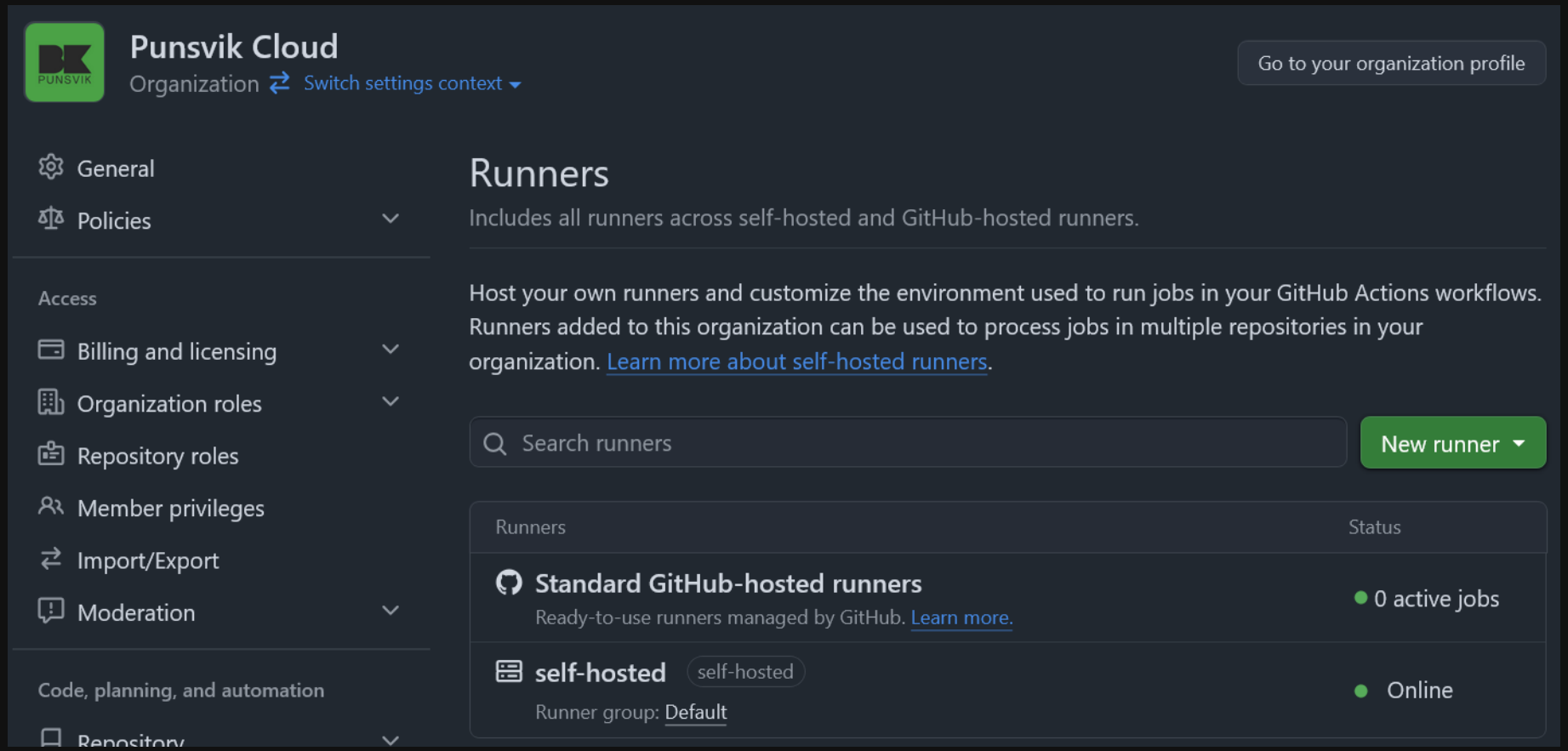

Communication, Collaboration and Security
Code
Plan
Deploy
Build
Dev
Release
Ops
Operate
Test
Monitor
Continuous Integration (CI)
Continuous Deployment (CD)
Continuous Delivery (CD)
Continuous Feedback (CF)


GitHub Actions

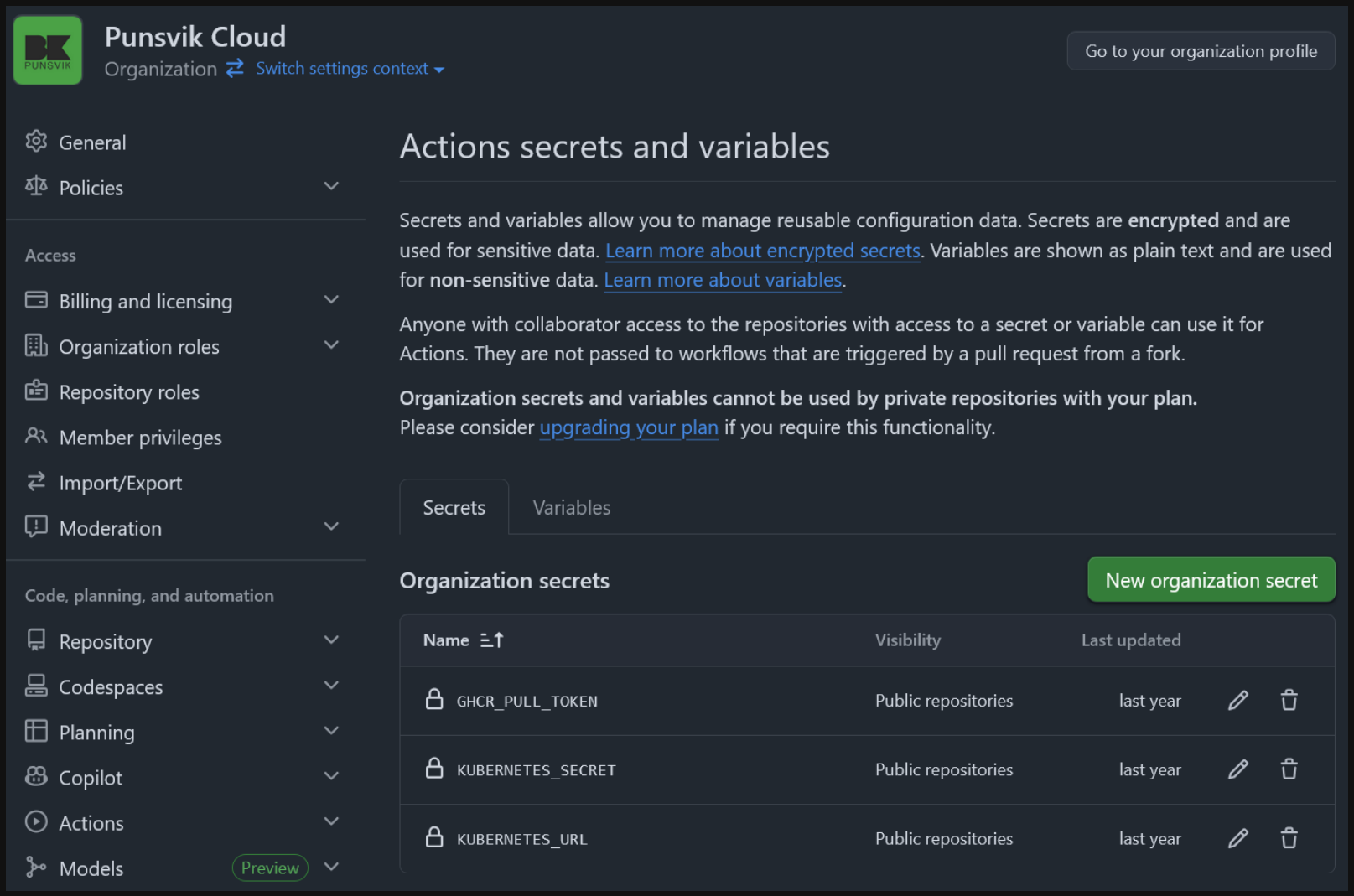
GitHub Action Runners



GitHub Action - Runners



GitHub Action - Secrets



DEVOPS

GitHub Action

`.github/workflows/deploy.yml` - **Metadata**

```yaml
name: Docker Build & Kubernetes Run

on:
  push:
    branches:
      - main
  workflow_dispatch:

env:
  REGISTRY: ghcr.io
  PLATFORMS: linux/arm64
  K8S_NAMESPACE: crayon

jobs:
  build:
    runs-on: self-hosted
    permissions:
      contents: read
      packages: write
      attestations: write
      id-token: write
    steps:
      ...
```

GitHub Action

.github/workflows/deploy.yml - **Build**

```yaml
- name: Checkout Code
  uses: actions/checkout@v4
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3
  with:
    driver: kubernetes
- name: Docker Login
  uses: docker/login-action@v3.3.0
  with:
    registry: ${{ env.REGISTRY }}
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN  }}
    logout: true
```

```yaml
- name: Extract metadata (tags, labels) for Docker
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${{ env.REGISTRY }}/${{ github.repository }}
    flavor: latest=true
    tags: |
      type=ref,event=branch
      type=raw,value=latest,enable={{is_default_branch}}
      type=raw,value=${{ github.sha }}
- name: Build and push Docker image
  id: push
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    platforms: linux/arm64
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
```

GitHub Action

.github/workflows/deploy.yml - **Deploy**

```yaml
deploy:
  runs-on: self-hosted
  needs: build
  permissions:
    id-token: write
    actions: read
    contents: read
  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Log in to the Container registry
      uses: docker/login-action@v3
      with:
        registry: ${{ env.REGISTRY }}
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN  }}

    - name: Set K8s context
      uses: azure/k8s-set-context@v4
      with:
        method: service-account
        k8s-url: ${{ secrets.KUBERNETES_URL }}
        k8s-secret: ${{ secrets.KUBERNETES_SECRET }}
```

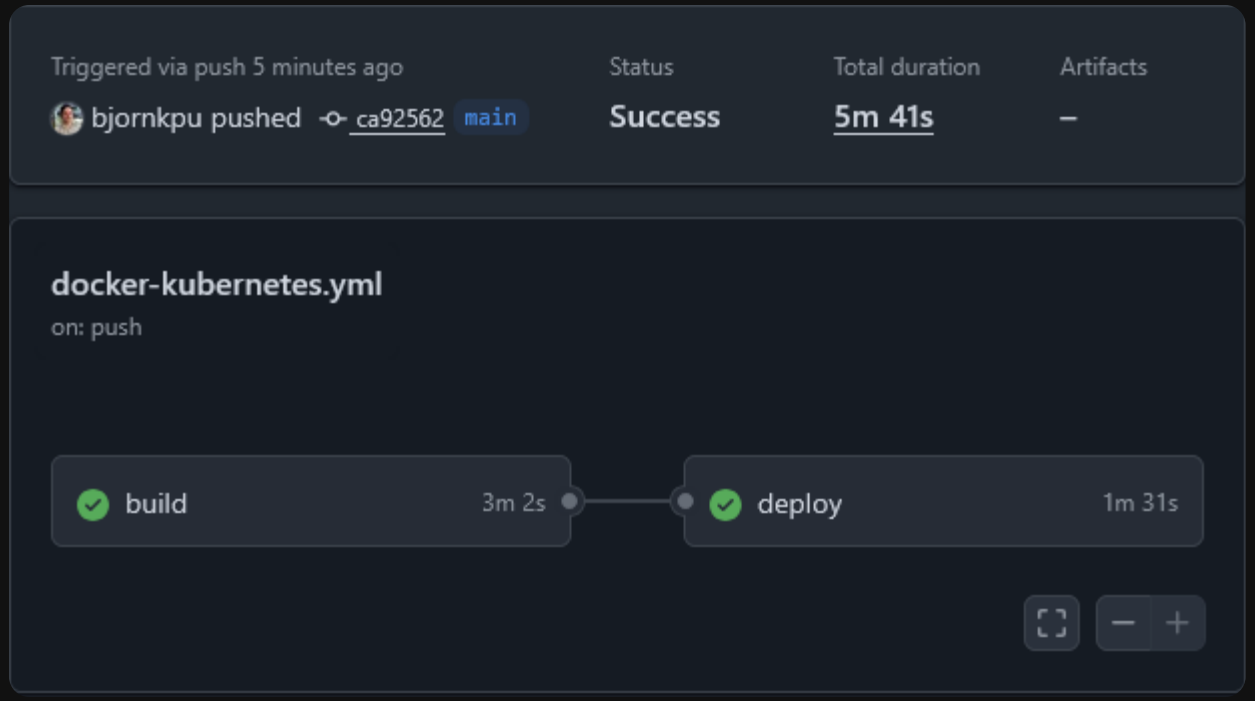
```yaml
  - name: Set kubectl
    uses: azure/setup-kubectl@v4
    id: install
    with:
      version: "v1.29.9"
  - name: Set imagePullSecret
    uses: azure/k8s-create-secret@v4
    id: create-secret
    with:
      namespace: ${{ env.K8S_NAMESPACE }}
      secret-name: regcred
      container-registry-url: ${{ env.REGISTRY }}
      container-registry-username: ${{ github.actor }}
      container-registry-password: ${{ secrets.GHCR_PULL_TOKEN }}
  - name: Set Label
    run: sed -i'' -e 's/IMAGE_LABEL/${{ github.sha }}/g' k8s.yml
  - name: Deploy application
    uses: azure/k8s-deploy@v5
    with:
      action: deploy
      manifests: k8s.yml
      namespace: ${{ env.K8S_NAMESPACE }}
      images: ${{ env.REGISTRY }}/${{ github.repository }}:${{ gith
```

Action waiting

```
Requested labels: self-hosted
Job defined at: PunsvikCloud/build-your-own-cloud/.github/workflows/docker-kubernetes.yml@refs/heads/main
Waiting for a runner to pick up this job...
Job is about to start running on the runner: self-hosted
```

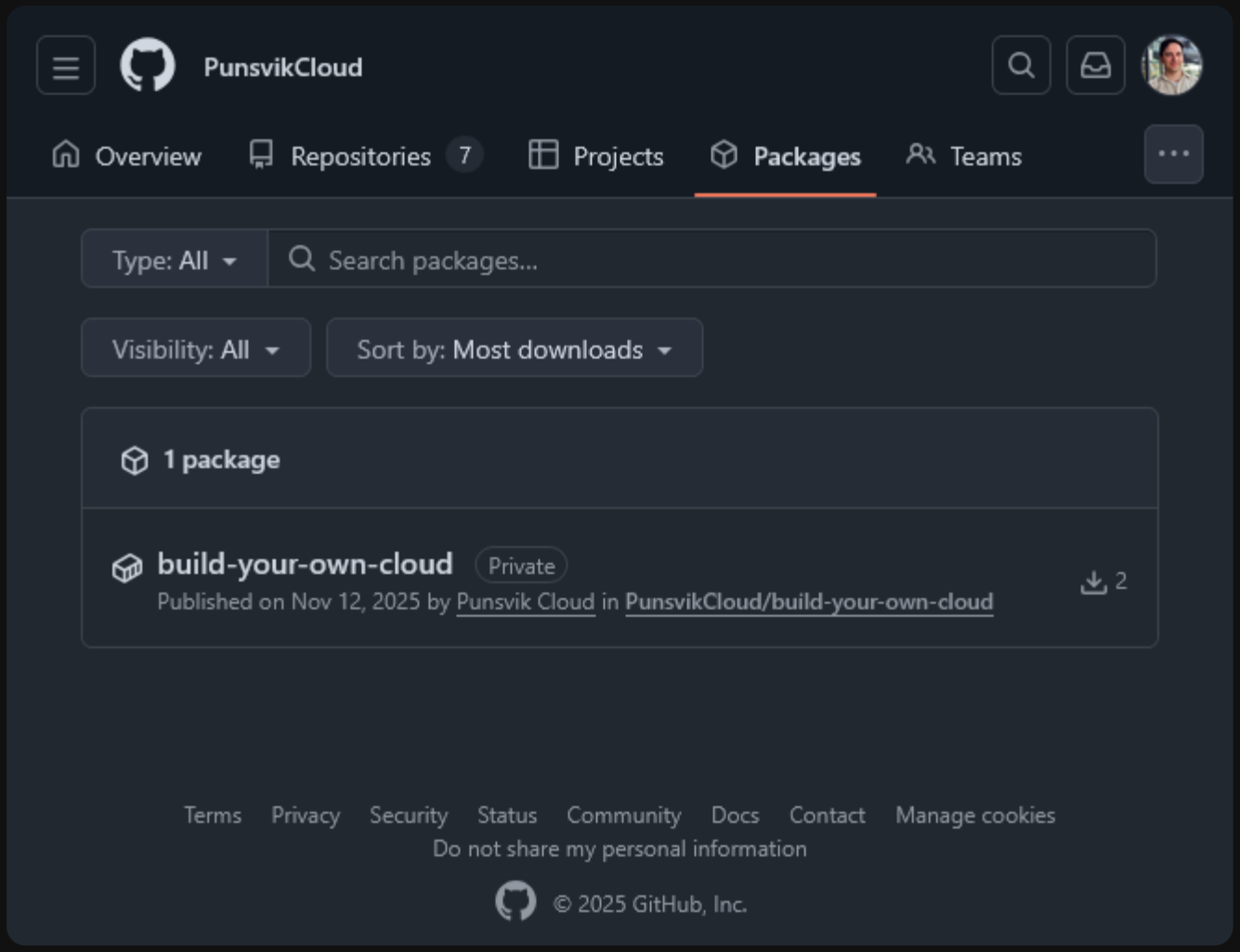
DEVOPS

Action details

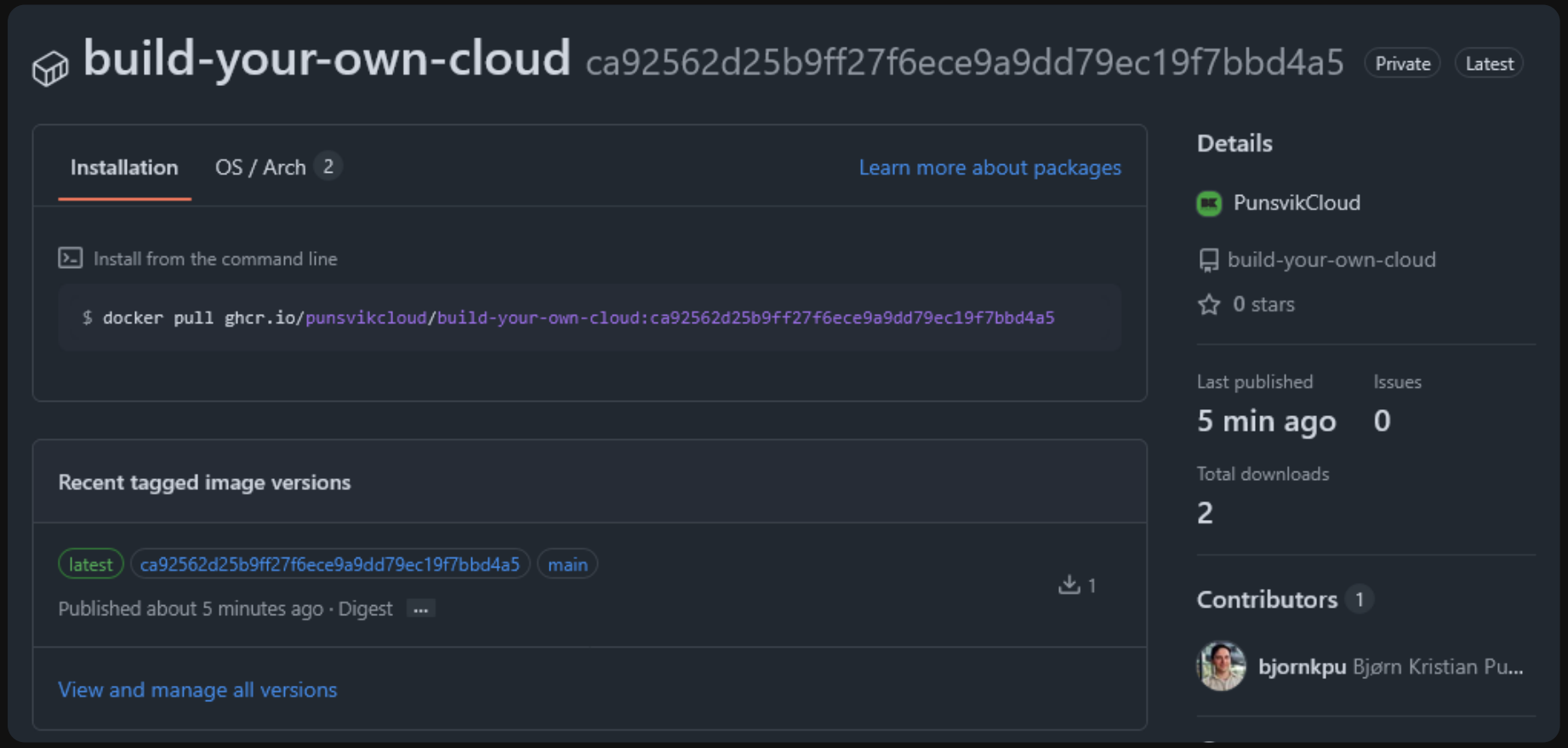


DEVOPS

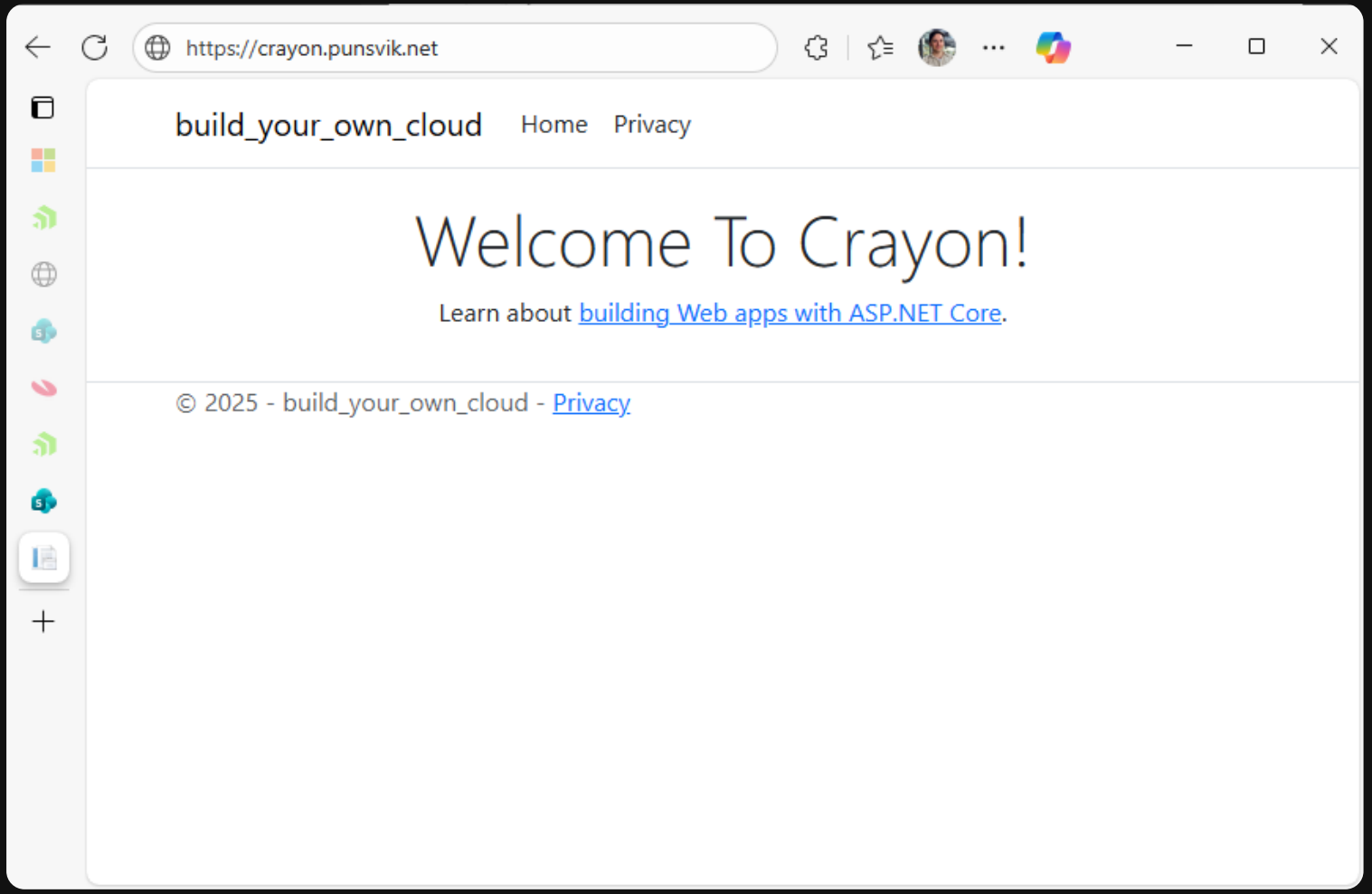
Packages



Package Details



Website


https://crayon.punsvik.net
build_your_own_cloud
Home
Privacy
Welcome To Crayon!
Learn about building Web apps with ASP.NET Core.
© 2025 - build_your_own_cloud - Privacy


DEVOPS